Multi-Domain 6G Network Slice Orchestration

Part I — Problem Formulation

VNF Placement, Resource Allocation, and Routing under Multi-Domain Infrastructure

1 System Model

1.1 Multi-Domain Physical Substrate

We consider a single-operator infrastructure partitioned into M administrative domains (e.g. geographically distinct zones, or different network tiers such as RAN-edge, MEC, and cloud). Each domain $m \in \{1, \ldots, M\}$ owns a subgraph $\mathcal{G}^m = (\mathcal{N}^m, \mathcal{L}^m)$, where $\mathcal{N}^m$ is its set of compute nodes and $\mathcal{L}^m$ its set of intra-domain transport links. The domains are interconnected by a set of inter-domain links $\mathcal{L}^{\text{inter}}$, yielding the full substrate graph:

$$G^P \;=\; \big(\mathcal{N},\; \mathcal{L}\big), \qquad \mathcal{N} = \bigcup_{m=1}^{M} \mathcal{N}^m, \qquad \mathcal{L} = \bigcup_{m=1}^{M} \mathcal{L}^m \;\cup\; \mathcal{L}^{\text{inter}}. \tag{1}$$

Each compute node $n \in \mathcal{N}$ is characterised by:

- CPU capacity C_n (vCPU units) and RAM capacity R_n (GB),
- infrastructure tier $\tau_n \in \{\text{RAN-edge},\ \text{MEC},\ \text{regional-cloud},\ \text{central-cloud}\}$,
- processing delay profile $\delta_{f,n} \geq 0$ (ms): the latency incurred when VNF f executes on node n, determined jointly by VNF computational intensity and server hardware class. It is obtained from offline profiling and stored as a matrix $\Delta \in \mathbb{R}_{\geq 0}^{|\mathcal{F}|\times|\mathcal{N}|}$ (see Remark 1),
- domain membership $m(n) \in \{1, \ldots, M\}$.

Each link $\ell \in \mathcal{L}$ is characterised by bandwidth capacity B_ℓ (Mbps), propagation delay D_ℓ (ms), and type $\theta_\ell \in \{\text{intra, inter}\}$, where inter-domain links carry a higher cost coefficient in the objective.

Definition 1 (Substrate graph)**.** *The physical substrate is a directed weighted graph $G^P = (\mathcal{N}, \mathcal{L})$ with node attributes $(C_n, R_n, \tau_n, \delta_{f,n}, m(n))$ and link attributes $(B_\ell, D_\ell, \theta_\ell)$.*

Remark 1 (Processing delay as a substrate attribute)**.** *$\delta_{f,n}$ is a first-class attribute of the substrate, not of the slice request. Under light load it is assumed constant (load-independent); a load-dependent extension is deferred to future work. This definition makes constraint C7 fully well-posed, as every term in the E2E delay sum has a known input value at solve time.*

1.2 Slice Request Model

A network slice request s is a tuple:

$$s \;=\; \big(\mathcal{F}_s,\; \mathcal{E}_s,\; \mathbf{q}_s,\; \mathcal{D}_s\big), \tag{2}$$

where:

- $\mathcal{F}_s = \{f_1^s, \ldots, f_{K_s}^s\}$ is an ordered chain of Virtual Network Functions (VNFs) forming the Service Function Chain (SFC),
- $\mathcal{E}_s \subseteq \mathcal{F}_s \times \mathcal{F}_s$ are directed flow edges between consecutive VNFs; each edge (f_k^s, f_{k+1}^s) carries a minimum bandwidth demand $\beta_{k,k+1}^s$,
- $\mathbf{q}_s = \left(D_s^{\max},\ \beta_s^{\min}\right)$ is the QoS vector specifying the maximum end-to-end (E2E) delay and the minimum per-flow throughput guarantee (see Remark 2),
- $\mathcal{D}_s$ encodes placement rules: for each VNF f_k^s, the set $\mathcal{D}_{f_k^s} \subseteq \mathcal{N}$ is the set of permitted nodes (tier and domain constraints).

Each VNF f_k^s has resource demands $(\tilde{c}_{f_k^s},\ \tilde{r}_{f_k^s})$ — the minimum CPU and RAM required for correct operation.

Remark 2 (QoS vector scope)**.** *The QoS vector is restricted to delay and throughput: the two dimensions for which hard constraints are formulated in Section 2.2. Reliability requires path-diversity or redundancy mechanisms not modelled in the current substrate formulation and is therefore excluded; it is deferred to future work.*

Definition 2 (Slice request)**.** *A slice request $s = (\mathcal{F}_s, \mathcal{E}_s, \mathbf{q}_s, \mathcal{D}_s)$ specifies an ordered SFC to be embedded onto G^P subject to QoS and placement constraints.*

1.3 Dual Arrival Model

The system is studied under two complementary settings that serve distinct roles in the overall framework.

Static offline setting (MILP oracle). A batch of S slice requests $\mathcal{S} = \{s_1, \ldots, s_S\}$ arrives simultaneously at $t = 0$ and is solved to global optimality by the MILP. This setting produces ground-truth solutions used as evaluation benchmarks and as few-shot examples for the LLM generator.

Dynamic online setting (RL deployment). Slice requests arrive sequentially as a Poisson process with rate λ. Each admitted slice s holds its resources for a stochastic lifetime $T_s \sim \mathrm{Exp}(\mu)$, after which it departs and releases all allocated resources. The system must make an immediate placement decision at each arrival event, without solving the full MILP. This is the target operational setting.

The two settings are complementary: the static setting provides the optimality benchmark that shapes RL training; the dynamic setting is the regime in which the trained policy is deployed.

2 Problem Formulation

2.1 Decision Variables

For each slice $s \in \mathcal{S}$, VNF $f_k^s \in \mathcal{F}_s$, node $n \in \mathcal{N}$, and link $\ell \in \mathcal{L}$, we define the decision variables in Table 1.

Table 1: Decision variables.

Variable	Domain	Semantics
$x_{f_k^s,n}$	$\{0,1\}$	VNF f_k^s is placed on node n
$a^c_{f_k^s,n}$	$\mathbb{R}_{\geq 0}$	CPU (vCPU) allocated to f_k^s on node n
$a^r_{f_k^s,n}$	$\mathbb{R}_{\geq 0}$	RAM (GB) allocated to f_k^s on node n
y_{ℓ,f_k^s,f_{k+1}^s}	$\{0,1\}$	Link ℓ carries the flow $(f_k^s \to f_{k+1}^s)$
$a^b_{\ell,f_k^s,f_{k+1}^s}$	$\mathbb{R}_{\geq 0}$	Bandwidth (Mbps) on ℓ for flow $(f_k^s \to f_{k+1}^s)$
z_s	$\{0,1\}$	Slice s is admitted

2.2 Constraints

C1 — Placement uniqueness. Each VNF of an admitted slice is assigned to exactly one node:

$$\sum_{n\in\mathcal{N}} x_{f_k^s,n} = z_s \qquad \forall\, s \in \mathcal{S},\ \forall\, f_k^s \in \mathcal{F}_s. \tag{3}$$

C2 — Node CPU capacity. Total CPU allocated across all co-located VNFs cannot exceed node capacity:

$$\sum_{s\in\mathcal{S}} \sum_{f_k^s\in\mathcal{F}_s} a^c_{f_k^s,n} \cdot x_{f_k^s,n} \ \leq\ C_n \qquad \forall\, n \in \mathcal{N}. \tag{4}$$

C3 — Node RAM capacity.

$$\sum_{s\in\mathcal{S}} \sum_{f_k^s\in\mathcal{F}_s} a^r_{f_k^s,n} \cdot x_{f_k^s,n} \ \leq\ R_n \qquad \forall\, n \in \mathcal{N}. \tag{5}$$

C4 — Resource sufficiency. Allocated resources must meet each VNF's minimum demand:

$$a^c_{f_k^s,n} \ \geq\ \tilde{c}_{f_k^s} \cdot x_{f_k^s,n}, \qquad a^r_{f_k^s,n} \ \geq\ \tilde{r}_{f_k^s} \cdot x_{f_k^s,n} \qquad \forall\, f_k^s,\, n. \tag{6}$$

C5 — Link bandwidth capacity.

$$\sum_{s\in\mathcal{S}} \sum_{k=1}^{K_s-1} a^b_{\ell,f_k^s,f_{k+1}^s} \cdot y_{\ell,f_k^s,f_{k+1}^s} \ \leq\ B_\ell \qquad \forall\, \ell \in \mathcal{L}. \tag{7}$$

C5b — Per-flow throughput floor. For each admitted slice, the bandwidth reserved on every flow must meet the minimum throughput declared in $\mathbf{q}_s$:

$$\sum_{\ell\in\mathcal{L}} a^b_{\ell,f_k^s,f_{k+1}^s} \cdot y_{\ell,f_k^s,f_{k+1}^s} \ \geq\ \beta_s^{\min} \cdot z_s \qquad \forall\, s \in \mathcal{S},\ \forall\, (f_k^s, f_{k+1}^s) \in \mathcal{E}_s. \tag{8}$$

C6 — Flow conservation. For each inter-VNF flow, the routing variables must form a valid directed path from the node hosting f_k^s to the node hosting f_{k+1}^s. Let $\delta^+(n)$ and $\delta^-(n)$ denote the sets of outgoing and incoming links of node n, respectively:

$$\sum_{\ell\in\delta^+(n)} y_{\ell,f_k^s,f_{k+1}^s} - \sum_{\ell\in\delta^-(n)} y_{\ell,f_k^s,f_{k+1}^s} \ =\ x_{f_k^s,n} - x_{f_{k+1}^s,n} \qquad \forall\, n \in \mathcal{N},\ \forall\, (f_k^s, f_{k+1}^s) \in \mathcal{E}_s. \tag{9}$$

C7 — End-to-end delay budget. The sum of propagation delays along all routed inter-VNF paths plus processing delays at hosting nodes must not exceed the slice's delay budget:

$$\sum_{k=1}^{K_s-1} \left(\sum_{\ell \in \mathcal{L}} D_\ell \cdot y_{\ell, f_k^s, f_{k+1}^s} + \sum_{n \in \mathcal{N}} \delta_{f_k^s, n} \cdot x_{f_k^s, n} \right) \leq D_s^{\max} \cdot z_s \qquad \forall\, s \in \mathcal{S}. \tag{10}$$

C8 — Placement rules. Each VNF may only be placed on nodes within its permitted set:

$$x_{f_k^s, n} = 0 \qquad \text{if } n \notin \mathcal{D}_{f_k^s}. \tag{11}$$

C9 — Inter-domain hop limit (optional, for tractability). A routing path may traverse at most H inter-domain links, reflecting operator routing policies:

$$\sum_{\ell \in \mathcal{L}^{\text{inter}}} y_{\ell, f_k^s, f_{k+1}^s} \leq H \qquad \forall\, (f_k^s, f_{k+1}^s) \in \mathcal{E}_s,\ \forall\, s. \tag{12}$$

Remark 3 (C7–C9 interaction)**.** *When both C7 and C9 are active simultaneously, their combined routing restrictions may render a slice infeasible even when each constraint alone would admit a valid path: the shortest-delay route may require more than H inter-domain hops. The MILP solver handles this automatically. At inference time, the feasibility checker detects joint infeasibility and triggers replanning.*

2.3 Objective Function

The objective maximises the number of admitted slices while penalising resource over-provisioning and expensive inter-domain bandwidth usage:

$$\max_{\mathbf{x}, \mathbf{a}, \mathbf{y}, \mathbf{z}} \quad \mu \sum_{s \in \mathcal{S}} z_s - \alpha \sum_{s, f_k^s, n} \left(a^c_{f_k^s, n} + a^r_{f_k^s, n} \right) x_{f_k^s, n} - \gamma^{\text{intra}} \sum_{\ell \in \mathcal{L}^{\text{intra}}} a^b_\ell - \gamma^{\text{inter}} \sum_{\ell \in \mathcal{L}^{\text{inter}}} a^b_\ell, \tag{13}$$

with $\mu \gg \alpha, \gamma$ enforcing that slice admission always takes priority over resource efficiency, and $\gamma^{\text{inter}} > \gamma^{\text{intra}}$ reflecting the higher cost of inter-domain bandwidth.

Remark 4 (Weight dominance)**.** *The condition $\mu \gg \alpha, \gamma$ is satisfied by setting*

$$\mu > \alpha \cdot S \cdot \max_s K_s \cdot \max_n (C_n + R_n) + \gamma^{\text{inter}} \cdot |\mathcal{L}| \cdot \max_\ell B_\ell,$$

which is computable from substrate parameters and guarantees lexicographic priority for admission over efficiency across all feasible instances.

Problem Statement 1 (Static VNF Embedding — MILP)**.** *Given the substrate G^P, a batch of slice requests $\mathcal{S}$, and placement rules $\{\mathcal{D}_s\}_s$, find the assignment $(\mathbf{x}, \mathbf{a}^c, \mathbf{a}^r, \mathbf{y}, \mathbf{a}^b, \mathbf{z})$ that maximises (13) subject to constraints (3)–(12).*

Problem 1 is a **Mixed-Integer Linear Program (MILP)**, which is NP-hard in general (it subsumes the VNF embedding problem, a generalisation of bin-packing and multi-commodity flow). The number of binary variables grows as $O(|\mathcal{F}| \cdot |\mathcal{N}| + |\mathcal{L}| \cdot |\mathcal{E}_s|)$ per slice, making exact solving intractable at operational timescales for realistic substrate sizes. This intractability motivates the learned online solution described in Part II.